Ex.No:9(a) IMPROVE THE REUSABILITY AND MAINTAINABILITY OF THE SOFTWARE SYSTEM BY APPLYING APPROPRIATE DESIGN PATTERNS – M.SUGANYA - 8649

EXAM REGISTRATION SYSTEM

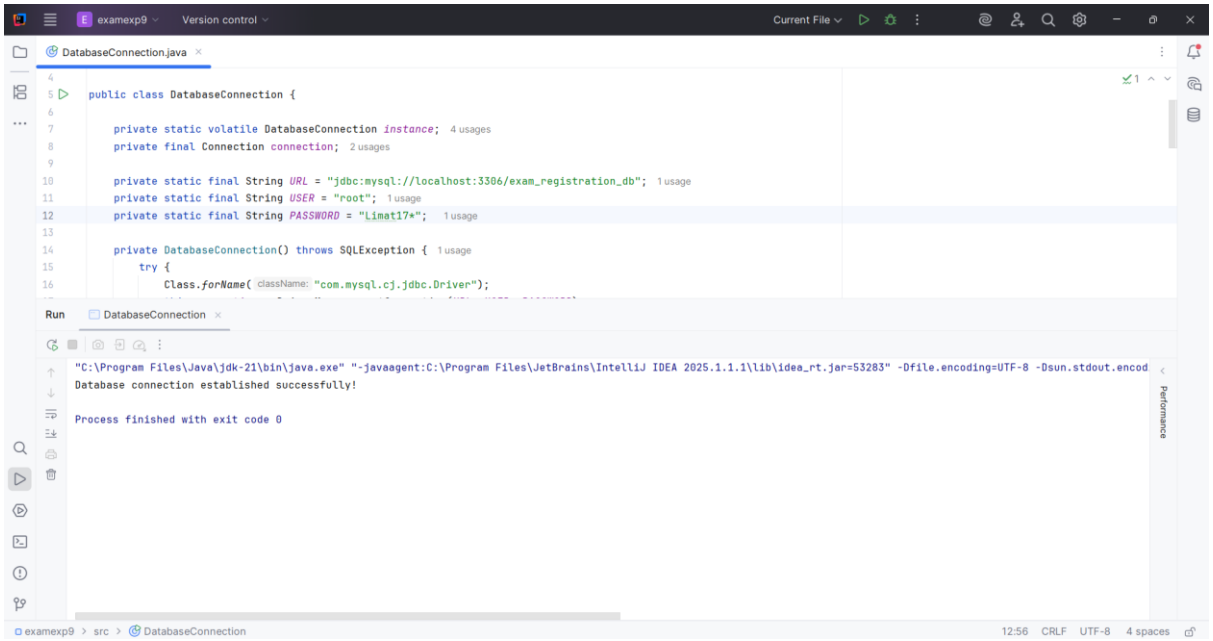
Identify Design Patterns:

- Singleton: Database Connection
- Factory Pattern: Object Creation
- Observer Pattern: Notifications

1. Singleton Pattern – DB Connection

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class DatabaseConnection {
    private static volatile DatabaseConnection instance;
    private final Connection connection;
    private static final String URL = "jdbc:mysql://localhost:3306/exam_registration_db";
    private static final String USER = "root";
    private static final String PASSWORD = "Limat17*";
    private DatabaseConnection() throws SQLException {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            this.connection = DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (ClassNotFoundException e) {
            throw new SQLException("MySQL JDBC Driver not found", e);
        }
    }
    public static DatabaseConnection getInstance() throws SQLException {
        if (instance == null) {
            synchronized (DatabaseConnection.class) {
                if (instance == null) {
                    instance = new DatabaseConnection();
                }
            }
        }
        return instance;
    }
    public Connection getConnection() {
        return connection;
    }
    public static void main(String[] args) {
        try {
            Connection conn = DatabaseConnection.getInstance().getConnection();
            if (conn != null && !conn.isClosed()) {
                System.out.println("Database connection established successfully!");
```

```
        } else {
            System.out.println("Failed to establish database connection.");
        }
    } catch (SQLException e) {
        System.err.println("Database error: " + e.getMessage());
    }
  }
}
```

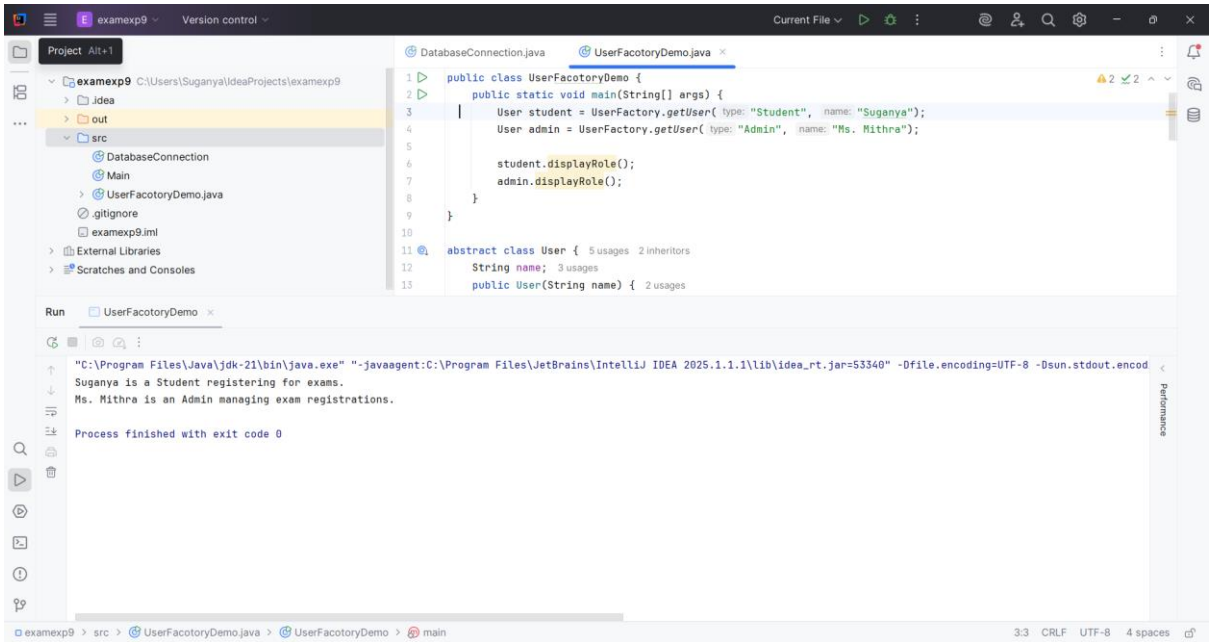


2. Factory Pattern – User Creation

```
public class UserFacotoryDemo {
    public static void main(String[] args) {
        User student = UserFactory.getUser("Student", "Suganya");
        User admin = UserFactory.getUser("Admin", "Ms. Mithra");

        student.displayRole();
        admin.displayRole();
    }
}
abstract class User {
    String name;
    public User(String name) {
        this.name = name;
    }
    public abstract void displayRole();
}
class StudentUser extends User {
    public StudentUser(String name) {
        super(name);
```

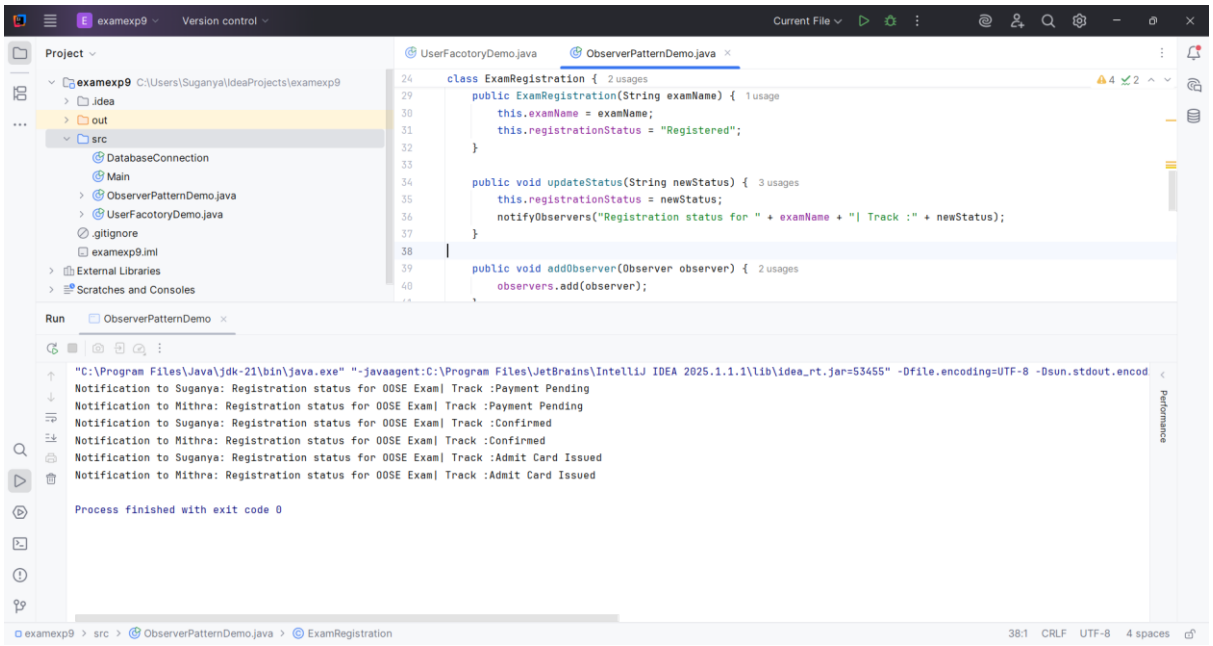
```
    }
    @Override
    public void displayRole() {
        System.out.println(name + " is a Student registering for exams.");
    }
}
class AdminUser extends User {
    public AdminUser(String name) {
        super(name);
    }
    @Override
    public void displayRole() {
        System.out.println(name + " is an Admin managing exam registrations.");
    }
}
class UserFactory {
    public static User getUser(String type, String name) {
        if (type.equalsIgnoreCase("Student")) {
            return new StudentUser(name);
        } else if (type.equalsIgnoreCase("Admin")) {
            return new AdminUser(name);
        } else {
            return null;
        }
    }
}
```



3.Observer Pattern

```
import java.util.ArrayList;
import java.util.List;
interface Observer {
    void update(String message);
}
class StudentObserver implements Observer {
    private String name;
    public StudentObserver(String name) {
        this.name = name;
    }
    @Override
    public void update(String message) {
        System.out.println("Notification to " + name + ": " + message);
    }
}
class ExamRegistration {
    private List<Observer> observers = new ArrayList<>();
    private String examName;
    private String registrationStatus;
    public ExamRegistration(String examName) {
        this.examName = examName;
        this.registrationStatus = "Registered";
    }
    public void updateStatus(String newStatus) {
        this.registrationStatus = newStatus;
        notifyObservers("Registration status for " + examName + "| Track :" + newStatus);
    }
    public void addObserver(Observer observer) {
        observers.add(observer);
    }
    private void notifyObservers(String message) {
        for (Observer o : observers) {
            o.update(message);
        }
    }
}
public class ObserverPatternDemo {
    public static void main(String[] args) {
        ExamRegistration registration = new ExamRegistration("OOSE Exam");
        Observer student1 = new StudentObserver("Suganya");
        Observer student2 = new StudentObserver("Mithra");
        registration.addObserver(student1);
        registration.addObserver(student2);
        registration.updateStatus("Payment Pending");
        registration.updateStatus("Confirmed");
```

```
        registration.updateStatus("Admit Card Issued");
    }
}
```



Ex.No:9(b) IMPROVE THE REUSABILITY AND MAINTAINABILITY OF THE SOFTWARE SYSTEM BY APPLYING APPROPRIATE DESIGN PATTERNS – M.SUGANYA - 8649

BOOK BANK:

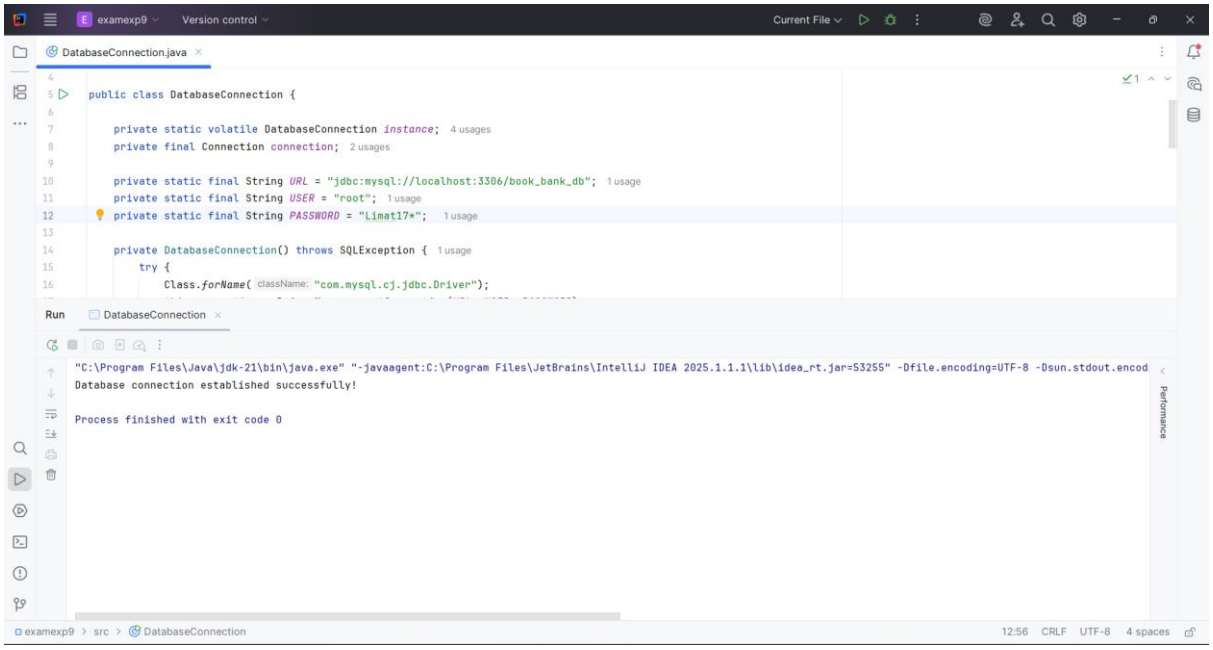
Identify Design Patterns:

- Singleton: Database Connection
- Factory Pattern: Object Creation
- Observer Pattern: Notifications

1. Singleton Pattern (Database Connection)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class DatabaseConnection {
    private static volatile DatabaseConnection instance;
    private final Connection connection;
    private static final String URL = "jdbc:mysql://localhost:3306/book_bank_db";
    private static final String USER = "root";
    private static final String PASSWORD = "Limat17*";
    private DatabaseConnection() throws SQLException {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            this.connection = DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (ClassNotFoundException e) {
            throw new SQLException("MySQL JDBC Driver not found", e);
        }
    }
    public static DatabaseConnection getInstance() throws SQLException {
        if (instance == null) {
            synchronized (DatabaseConnection.class) {
                if (instance == null) {
                    instance = new DatabaseConnection();
                }
            }
        }
        return instance;
    }
    public Connection getConnection() {
        return connection;
    }
    public static void main(String[] args) {
        try {
            Connection conn = DatabaseConnection.getInstance().getConnection();
            if (conn != null && !conn.isClosed()) {
                System.out.println("Database connection established successfully!");
```

```
            } else {
                System.out.println("Failed to establish database connection.");
            }
        } catch (SQLException e) {
            System.err.println("Database error: " + e.getMessage());
        }
    }
}
```

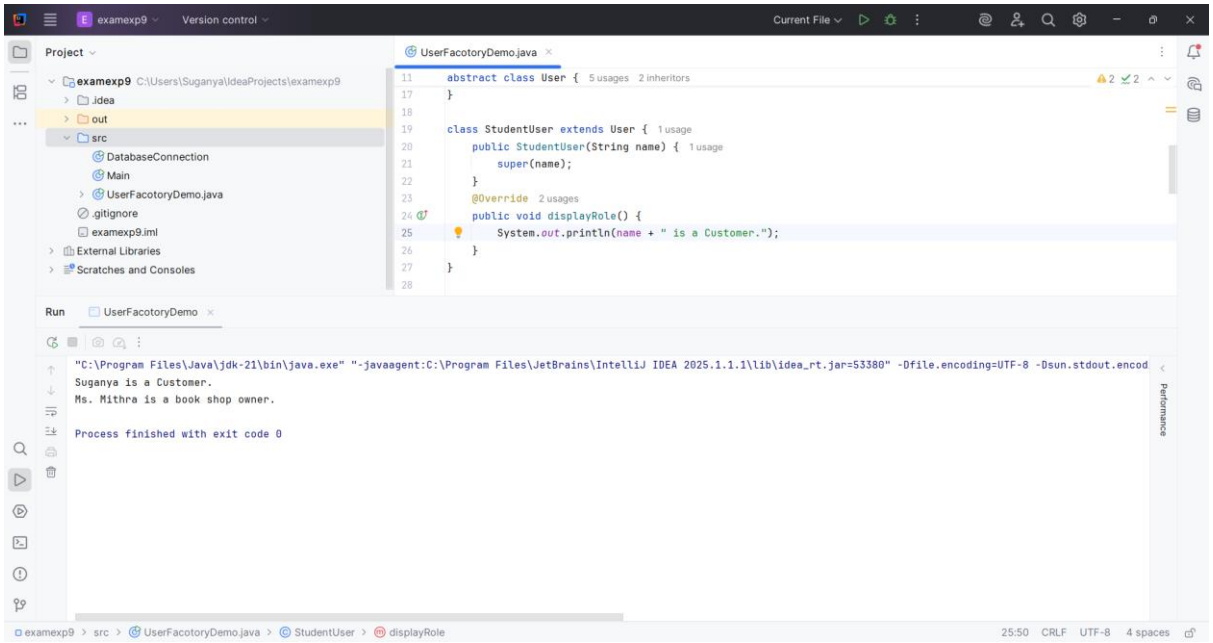


2.Factory Pattern

```
public class UserFacotoryDemo {
    public static void main(String[] args) {
        User student = UserFactory.getUser("Student", "Suganya");
        User admin = UserFactory.getUser("Admin", "Ms. Mithra");

        student.displayRole();
        admin.displayRole();
    }
}
abstract class User {
    String name;
    public User(String name) {
        this.name = name;
    }
    public abstract void displayRole();
}
class StudentUser extends User {
    public StudentUser(String name) {
```

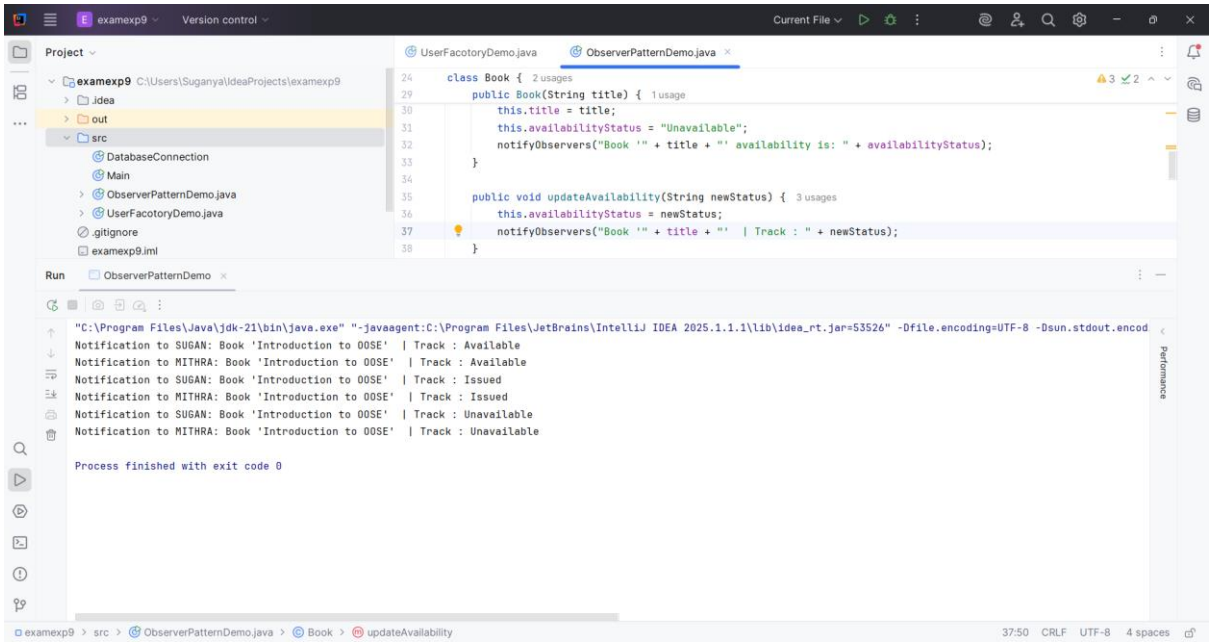
```
        super(name);
    }
    @Override
    public void displayRole() {
        System.out.println(name + " is a Customer.");
    }
}
class AdminUser extends User {
    public AdminUser(String name) {
        super(name);
    }
    @Override
    public void displayRole() {
        System.out.println(name + " is a book shop owner.");
    }
}
class UserFactory {
    public static User getUser(String type, String name) {
        if (type.equalsIgnoreCase("Student")) {
            return new StudentUser(name);
        } else if (type.equalsIgnoreCase("Admin")) {
            return new AdminUser(name);
        } else {
            return null;
        }
    }
}
```



3. Observer Pattern – Book Availability Notifications

```
import java.util.ArrayList;
import java.util.List;
interface Observer {
    void update(String message);
}
class BookBankUser implements Observer {
    private String name;
    public BookBankUser(String name) {
        this.name = name;
    }
    @Override
    public void update(String message) {
        System.out.println("Notification to " + name + ": " + message);
    }
}
class Book {
    private List<Observer> observers = new ArrayList<>();
    private String title;
    private String availabilityStatus;
    public Book(String title) {
        this.title = title;
        this.availabilityStatus = "Unavailable";
        notifyObservers("Book '" + title + "' availability is: " + availabilityStatus);
    }
    public void updateAvailability(String newStatus) {
        this.availabilityStatus = newStatus;
        notifyObservers("Book '" + title + "'  | Track : " + newStatus);
    }
    public void addObserver(Observer observer) {
        observers.add(observer);
    }
    private void notifyObservers(String message) {
        for (Observer o : observers) {
            o.update(message);
        }
    }
}
public class ObserverPatternDemo {
    public static void main(String[] args) {
        Book book = new Book("Introduction to OOSE");
        Observer user1 = new BookBankUser("SUGAN");
        Observer user2 = new BookBankUser("MITHRA");
        book.addObserver(user1);
        book.addObserver(user2);
        book.updateAvailability("Available");
```

```
        book.updateAvailability("Issued");
        book.updateAvailability("Unavailable");
    }
}
```



Ex.No:9(c) IMPROVE THE REUSABILITY AND MAINTAINABILITY OF THE SOFTWARE SYSTEM BY APPLYING APPROPRIATE DESIGN PATTERNS – M.SUGANYA - 8649

STUDENT INFORMATION SYSTEM:

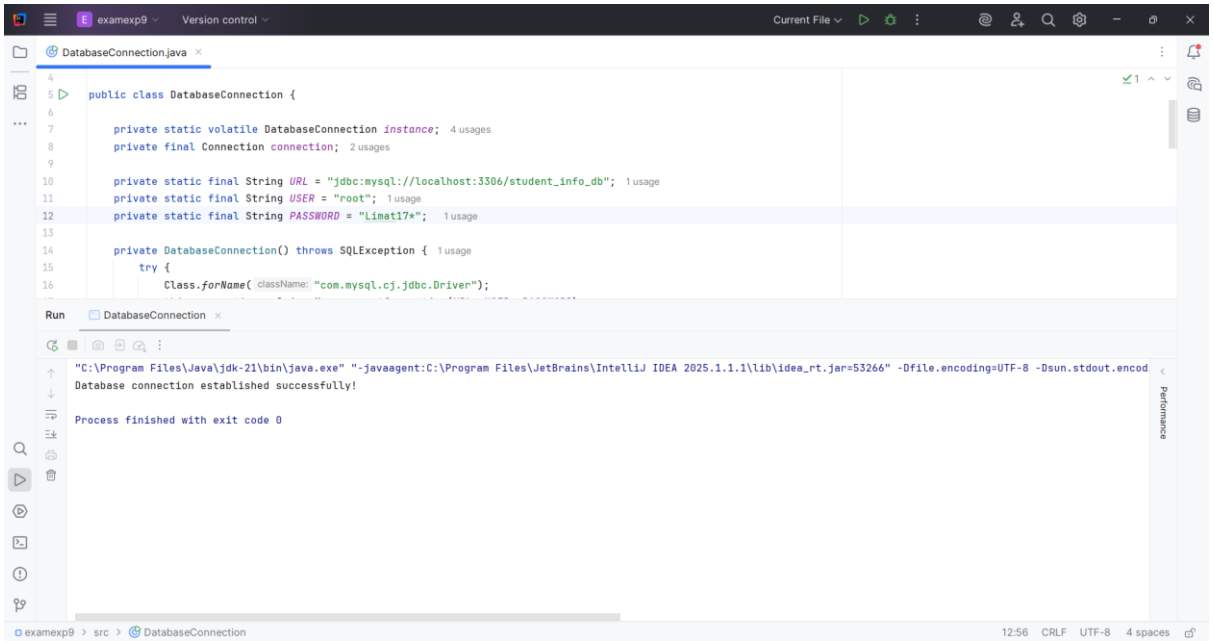
Identify Design Patterns:

- Singleton: Database Connection
- Factory Pattern: Object Creation
- Observer Pattern: Notifications

1. Singleton Pattern (Database Connection)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class DatabaseConnection {
    private static volatile DatabaseConnection instance;
    private final Connection connection;
    private static final String URL = "jdbc:mysql://localhost:3306/student_info_db";
    private static final String USER = "root";
    private static final String PASSWORD = "Limat17*";
    private DatabaseConnection() throws SQLException {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            this.connection = DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (ClassNotFoundException e) {
            throw new SQLException("MySQL JDBC Driver not found", e);
        }
    }
    public static DatabaseConnection getInstance() throws SQLException {
        if (instance == null) {
            synchronized (DatabaseConnection.class) {
                if (instance == null) {
                    instance = new DatabaseConnection();
                }
            }
        }
        return instance;
    }
    public Connection getConnection() {
        return connection;
    }
    public static void main(String[] args) {
        try {
            Connection conn = DatabaseConnection.getInstance().getConnection();
            if (conn != null && !conn.isClosed()) {
                System.out.println("Database connection established successfully!");
```

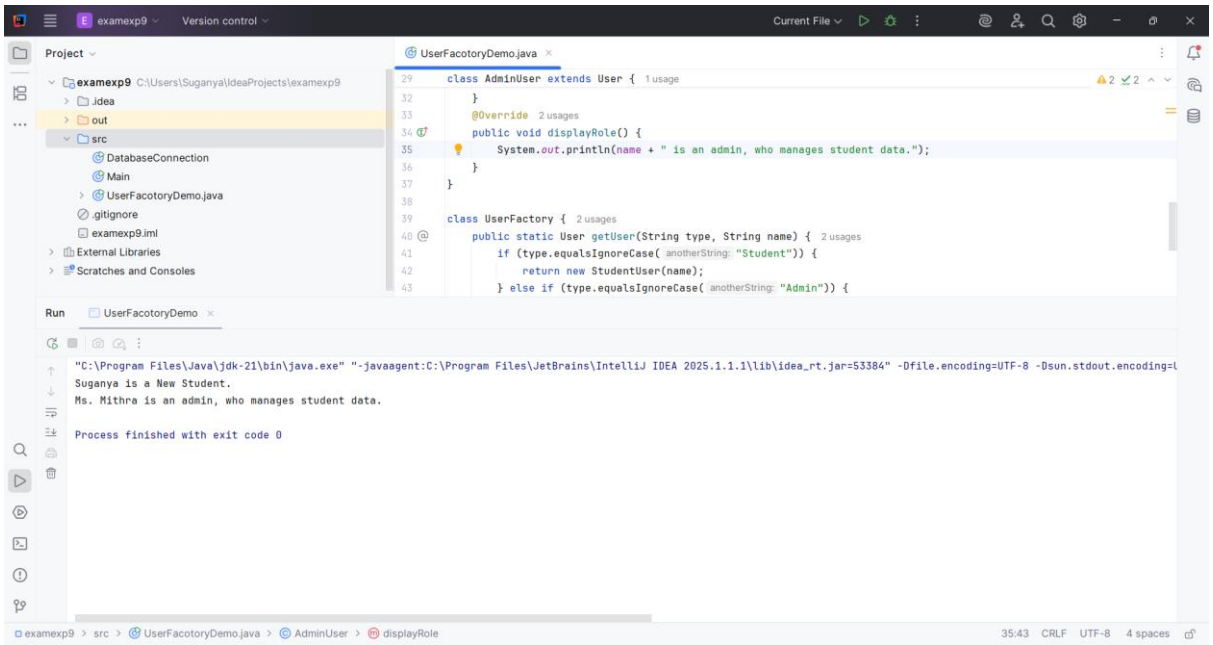
```
            } else {
                System.out.println("Failed to establish database connection.");
            }
        } catch (SQLException e) {
            System.err.println("Database error: " + e.getMessage());
        }
    }
}
```



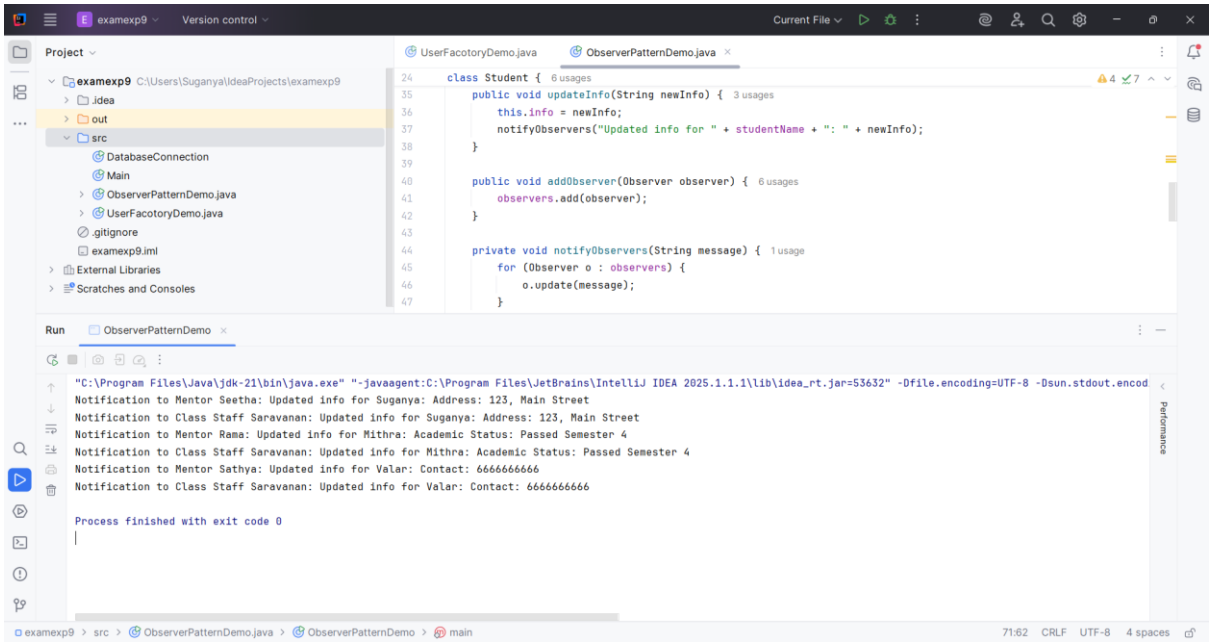
2. Factory Pattern

```
public class UserFacotoryDemo {
    public static void main(String[] args) {
        User student = UserFactory.getUser("Student", "Suganya");
        User admin = UserFactory.getUser("Admin", "Ms. Mithra");
        student.displayRole();
        admin.displayRole();
    }
}
abstract class User {
    String name;
    public User(String name) {
        this.name = name;
    }
    public abstract void displayRole();
}
class StudentUser extends User {
    public StudentUser(String name) {
        super(name);
    }
    @Override
```

```
    public void displayRole() {
        System.out.println(name + " is a New Student.");
    }
}
class AdminUser extends User {
    public AdminUser(String name) {
        super(name);
    }
    @Override
    public void displayRole() {
        System.out.println(name + " is an admin, who manages student data.");
    }
}
class UserFactory {
    public static User getUser(String type, String name) {
        if (type.equalsIgnoreCase("Student")) {
            return new StudentUser(name);
        } else if (type.equalsIgnoreCase("Admin")) {
            return new AdminUser(name);
        } else {
            return null;
        }
    }
}
```



3.Observer Pattern – Student Information Update Notifications

```
import java.util.ArrayList;
import java.util.List;
interface Observer {
    void update(String message);
}
class StaffObserver implements Observer {
    private String name;
    public StaffObserver(String name) {
        this.name = name;
    }
    @Override
    public void update(String message) {
        System.out.println("Notification to " + name + ": " + message);
    }
}
class Student {
    private List<Observer> observers = new ArrayList<>();
    private String studentName;
    private String info;
    public Student(String studentName, String info) {
        this.studentName = studentName;
        this.info = info;
    }
    public void updateInfo(String newInfo) {
        this.info = newInfo;
        notifyObservers("Updated info for " + studentName + ": " + newInfo);
    }
    public void addObserver(Observer observer) {
        observers.add(observer);
    }
    private void notifyObservers(String message) {
        for (Observer o : observers) {
            o.update(message);
        }
    }
}
public class ObserverPatternDemo {
    public static void main(String[] args) {
        Student student1 = new Student("Suganya", "Contact: 9999999999");
        Student student2 = new Student("Mithra", "Contact: 8888888888");
        Student student3 = new Student("Valar", "Contact: 7777777777");
        student1.addObserver(new StaffObserver("Mentor Seetha"));
        student1.addObserver(new StaffObserver("Class Staff Saravanan"));
        student2.addObserver(new StaffObserver("Mentor Rama"));
        student2.addObserver(new StaffObserver("Class Staff Saravanan"));
```

```
        student3.addObserver(new StaffObserver("Mentor Sathya"));
        student3.addObserver(new StaffObserver("Class Staff Saravanan"));
        student1.updateInfo("Address: 123, Main Street");
        student2.updateInfo("Academic Status: Passed Semester 4");
        student3.updateInfo("Contact: 6666666666");
    }
}
```